Dangerous Object Detection in X-ray Images with Deep Reinforcement Learning

Tran Ha Bao Long - SE182345 Bui Cong Bang - SE18 Phan Duc Nghia - SE18

Abstract

Localization of dangerous items in baggage X-ray remains difficult due to cluttered contents, occlusions, low material contrast, and high intra-class variability. We cast localization as an active decision process and propose a deep reinforcement learning agent that iteratively refines a bounding box and decides when to stop. A YOLOv9 (GELAN-c) backbone, pre-trained on generic data and *fine-tuned* for the X-ray domain, supplies compact visual features; a Deep Q-Network operates on these features and a short action history to select among nine geometric actions. A modular recognition head (YOLOv8) provides post-localization semantic verification while remaining decoupled from policy learning. We extend the single-instance setting with a simple iterative masking scheme to reveal multiple objects per scene. Through ablations on feature design and step budgets, the proposed active localization exhibits efficient, interpretable search with competitive accuracy and low inference cost. We conclude with limitations and directions for improvement.

I. Introduction

A. Scenario

Object classification and object detection are two of the most important tasks in computer vision. An object classification algorithm identifies which objects are present in an image, whereas an object detection (localization) algorithm not only indicates which objects are present, but also outputs bounding boxes to mark their locations. Object detection underpins numerous real-world applications: autonomous driving (cars, pedestrians, traffic lights, road signs), multi-object tracking for traffic monitoring and activity recognition, sports analytics, surveillance and security, face and iris detection for identity verification, and medical imaging where organ detection assists diagnosis, therapy planning, and image-guided interventions. In security screening, detecting dangerous items in baggage X-ray is particularly challenging due to clutter, occlusion, low material contrast, and significant intra-class variation, all under strict throughput and compute constraints.

In this project we train a model to localize dangerous objects in baggage X-ray images using a deep reinforcement learning approach [1]. We formulate object localization as a Markov Decision Process (MDP) and let an agent sequentially focus on salient regions likely to contain a target, refining a bounding box over time until a tight localization is achieved. Concretely, the agent operates over a discrete action space with nine actions: eight geometric transformations (horizontal/vertical moves, scale changes, and aspect-ratio changes) and a stop action. Each geometric update is applied with a fixed ratio, and episodes are capped by a small step budget, yielding efficient, interpretable search. The reward encourages monotonic improvement in Intersection-over-Union (IoU) with a terminal bonus/penalty based on the final IoU. To reveal multiple instances in a scene, we adopt an iterative masking strategy: after a localization, the region is attenuated with mean-plus-noise and the search is repeated to discover additional objects. For semantic confirmation, we include a modular, post-localization recognition head (YOLOv8) [4] that is decoupled from policy learning. Our policy builds on compact features extracted by a YOLOv9 (GELAN-c) backbone [3] pre-trained on generic data and fine-tuned for the X-ray domain; features are pooled via adaptive average pooling to a 512-dimensional vector and concatenated with a short one-hot action history for decision making via a Deep Q-Network. We adapted an open-source DRL implementation as a starting point [2] and extended it with these components for the X-ray domain.

B. Background

Modern object detection pipelines begin with extracting informative visual features using convolutional neural networks. Convolutions reduce spatial dimensions while preserving local structure, making them well suited to images. In our system, a YOLOv9 GELAN-c backbone provides multi-scale features that we pool into a compact 512-d representation; the backbone is initialized from large-scale pre-training and fine-tuned to mitigate X-ray domain shift.

We use DQN to estimate state–action values for the discrete action space. Given a state s and action a, the optimal action-value function $Q^*(s, a)$ satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} Q^*(s', a') \right], \tag{1}$$

where γ is the discount factor and the expectation is over the environment dynamics. DQN parameterizes $Q(s, a; w)$ with a neural network and minimizes the temporal-difference error between the current estimate and a target constructed from (1). A stochastic-gradient update akin to one-step Q-learning is

$$w_{t+1} = w_t + \alpha \left[R_{t+1} + \gamma \max_{a} Q(s_{t+1}, a; w_t) - Q(s_t, a_t; w_t) \right] \nabla_w Q(s_t, a_t; w_t), \tag{2}$$

with learning rate α. In practice, DQN uses experience replay to decorrelate updates (sampling mini-batches of transitions $e_t = (s_t, a_t, r_{t+1}, s_{t+1})$ from a replay buffer) and a slowly updated target network to stabilize learning. With these components, we can learn a policy that proposes a small number of box refinements to localize dangerous items effectively in X-ray images.

II. Related Works

A. DRL for active object localization

Active localization methods cast bounding-box adjustment as a sequential decision process. An agent observes visual features and executes discrete geometric transformations (translate, scale, aspect change), receiving feedback related to localization quality. Deep Q-Networks stabilize learning in such discrete spaces via replay and target networks and have been widely adopted for visual decision tasks. A representative approach is Caicedo and Lazebnik's active localization with DRL [1], which demonstrates iterative refinement under occlusion and clutter and motivates the MDP formulation used in our work.

B. YOLOv9 backbones

Recent YOLO variants, including YOLOv9 with its GELAN backbone [3], introduce improved feature hierarchies and training strategies, achieving strong speed–accuracy trade-offs. Using a YOLOv9 backbone as a frozen feature extractor yields compact, high-quality embeddings that are especially beneficial for low-contrast and cluttered X-ray imagery.

C. YOLOv8 classification

YOLOv8 provides a versatile classification head suitable for fast recognition on cropped regions [4]. In our pipeline, a YOLOv8 classifier trained on four dangerous categories (blade, gun, knife, shuriken) and a *no-object* background class verifies the semantic class of the localized region in a *post-localization, decoupled* manner. This separation of concerns (localize with DRL, recognize with a classifier) mitigates the issue that an RL agent is structurally compelled to output a bounding box, thereby reducing false positives under ambiguity.

III. Method

A. Markov Decision Process

The object localization problem can be viewed as a dynamic control process to transform the geometry and location of the bounding box with a sequence of steps, which can then be formulated as a dynamic Markov decision process. In this formulation, a single image will be viewed as the environment, and the states S are constructed based on the features of the current bounded pixels and a series of previous actions in history vector. The actions A are composed of the geometric transformations of the bounding box, and the reward function R is defined based on the effects of these transformation actions. We will further clarify each component of MDP formulation as follows.

1) States S: It is expected that the state space of this problem formulation should be very large due to the complexity of dealing with the image data. As a result, generalization with feature extraction is necessary to represent the states efficiently and effectively. In this project, the states are constructed with a feature vector o, which summarizes the image information of the current found bounding box, and a history vector h of previous taken actions. The feature vector o is extracted by feeding the current bounded region of the image into the fine-tuned YOLOv9 (GELAN-c) backbone [3]. The attended region is letterboxed to 640×640, passed through the backbone, and adaptively average-pooled to a 512-dimensional vector.

The action history vector h is a binary vector that encodes the actions applied in the past with one-hot encoding method. This vector stores n previous actions taken by the agent, and as we have 9 possible actions, the one-hot encoded history vector will have $9\times$n dimensions. By adding this history vector as part of the state representation, the agent is expected to be informed about what has happened in the past and avoid getting stuck in repetitive search cycles.

2) Action A: As stated above, the actions of this MDP are mainly the geometric transformations of the bounding boxes. There are nine transformation actions and one terminal action in this MDP formulation . The eight transformation actions can be divided into four categories, which modify the horizontal level, vertical level, scale and aspect ratio of the bounding box, respectively. A figure of all actions can be seen in figure 1. The transformation actions are executed by calculating a discrete change to the box by a relative factor α, using equations 3.

$$\alpha_w = \alpha(x_2 - x_1) \quad \alpha_h = \alpha(y_2 - y_1) \tag{3}$$

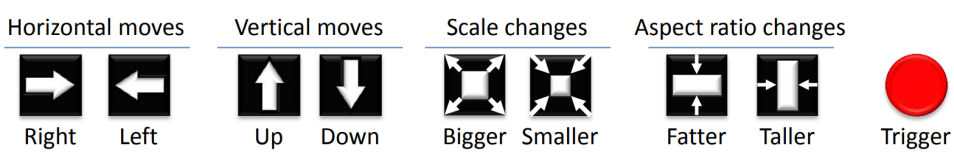

Fig. 1: Illustration of the actions in MDP. Image credit [1]

Where x_1, y_1, x_2 and y_2 are the coordinates of the top- left corner and the right-bottom corner of the current placed bounding box. With this calculated discrete change, the action can be carried out by adding this value to or subtract this value from the corresponding coordinates.

3) Intersection-over-Union (IoU): Before explaining the reward formulation of our MDP, we need to define a metric called Intersection-over-Union (IoU). We mainly exploit the Intersection over Union (IoU) between the detecting window and the ground truth mask to evaluate the performance of our model. The IoU is defined as the ratio of the overlapped area between the predicted bounding box and the ground-truth bounding box over the area encompassed by both of them. More specifically, given the predicted bounding box b and the ground truth bounding box g, the IoU between b and g is expressed as the following formula:

$$\mathrm{IoU}(b, g) = \frac{\mathrm{area}(b \cap g)}{\mathrm{area}(b \cup g)} \tag{4}$$

4) Reward function R: The reward function needs to be defined corresponding to the actions taken. In the proposed formulation, the reward function R is given based on the improvement of the Intersection-over-Union (IoU) between the predicted and the ground truth bounding box, during the transition between states. The reward function $R_a(s, s')$ is granted to the agent when it chooses the action a to move from state s to s'. Each state s has an associated box b that contains the attended region. For non-terminal actions, the reward is set to 1 if the state transition improves the IoU and -1 otherwise. as below formula:

$$R_a(s, s') = \begin{cases} +1, & \text{if } \mathrm{IoU}(b', g) > \mathrm{IoU}(b, g) \\ -1, & \text{otherwise} \end{cases} \tag{5}$$

For the terminal action (trigger), the reward scheme is different since the terminal action does not change the geometry of the bounding box. As a result, the reward is defined based on whether the current IoU exceeds the pre-set threshold (τ) or not. The terminal reward is set to be 3 if the current IoU is larger than the threshold and -3 otherwise, as the formula below:

$$R_w(s, s') = \begin{cases} +\eta, & \text{if } \mathrm{IoU}(b, g) \geq \tau \\ -\eta, & \text{otherwise} \end{cases} \tag{6}$$

Where μ was chosen to be 3 and τ (trigger threshold) was chosen to be 0.5.

Initially, we modified the reward function to:

$$R_a(s, s') = (\mathrm{IoU}(b', g) - \mathrm{IoU}(b, g)) \times 10 \tag{7}$$

$$R_w(s, s') = \begin{cases} +15, & \text{if } \mathrm{IoU}(b, g) > 0.8 \\ -15, & \text{if } \mathrm{IoU}(b, g) < 0.2 \\ -5, & \text{otherwise} \end{cases} \tag{8}$$

However, after training we observed that since each action updates the box with a small step size α (we set α=0.1), the transition reward from state s to s' did not differ much from the formulation above. In contrast, the terminal-action penalty was set too high, which caused the agent to receive mostly negative rewards at the end of each episode.

B. Model and implementation

The architecture we used for our DQN model can be seen in figure 2. This model is similar to the model proposed by authors in [1]. However, we add one more dense layer with 1024 units before the last layer for better average IoU score according the hyper-parameter tuning experiment in [2]. Addictionally, to extract features from each region, instead of using a 5-layer convolutional neural network, we use a YOLOv9 backbone fine-tuned on GDXray dataset. To feed each attended region to the backbone, we first letterbox the crop to 640×640 and normalize the input. The backbone produces multi-scale features that we aggregate with adaptive average pooling into a compact 512-dimensional descriptor. As in [1], we then concatenate this descriptor with a one-hot action history of the past L steps, we choose 1 due to the hyper-parameter tuning experiment in [2]. The resulting state vector is the input to our deep Q-network. The first three layers are composed of 1024 units and the last dense layer contains 9 units and outputs the q-values for each of the 9 actions.

Also to calculate the loss of our DQN, we used smooth L1 loss [2]. Smooth L1-loss can be interpreted as a combination of L1-loss and L2-loss. It behaves as L1-loss when the absolute value of the argument is high, and it behaves like L2-loss when the absolute value of the argument is close to zero.

However, we realize that an agent can only predict one bounding box and even if the image is not contained dangerous object, agent will not know it and keep the bounding box. Therefore, we investigate a way agent can predict more than one bounding box and classify the object using Yolov8 [4], but we will explain it in Inference section.

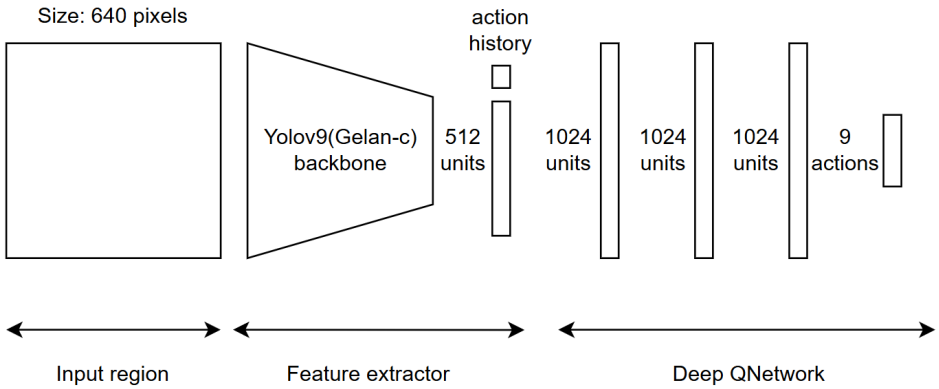

Fig. 2: Architecture of our Qnetwork

C. Dataset

Our experimental framework utilizes X-ray image data from the GDXray dataset:

1) YOLOv9 Backbone Pre-training: We use GDXray data from Roboflow(https://universe.roboflow.com/nust-university/gdxray-pqsih) with about 2909 images(70% train, 20% validate, 10% test), a widely-used platform for computer vision dataset management and augmentation. This dataset is already provided Yolov9 format so we do not need to preprocess it.

2) DRL Agent Training Data: We obtain additional GDXray data from Dataset Ninja(https://datasetninja.com/gdxray and perform custom preprocessing and clean, after that we having about 11000 images(80% train, 10% validate, 10% test). This dataset provides diverse X-ray scenes with various occlusion and clutter patterns, enabling the DQN agent to learn effective localization policies through reinforcement learning.

3) YOLOv8 Classifier Training Dataset: A curated subset derived from our baggage screening data. We manually crop X-ray regions containing dangerous objects and perform precise annotations for five classes: gun, knive, blade, shuriken, and no_object (miscellaneous dangerous items) and use augmentation(flip: horizontal, veritcal) to increase the data. This results in: https://universe.roboflow.com/long-otrie/dangerous-object-classification4-sdj9s/dataset/2

D. Training Step

We employ standard DQN training with experience replay and a target network to stabilize learning. The training procedure is outlined as follows:

1) Experience Replay: At each step, we store a transition tuple $(s_t, a_t, r_{t+1}, s_{t+1})$ in a replay buffer of maximum size $M_{\text{buffer}} = 1000$. We sample mini-batches of size $B = 20$ from the buffer and compute the temporal-difference error:

$$\mathcal{L}(w) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; w^-) - Q(s, a; w)\right)^2\right] \tag{9}$$

where w are the policy network parameters, w^- are the slowly updated target network parameters, $\gamma = 0.5$ is the discount factor, and $\mathcal{D}$ is the replay buffer. We use Smooth L1 Loss instead of MSE to improve training stability and robustness to outliers.

2) Epsilon-Greedy Exploration: During training, we follow an epsilon-greedy policy where:

- With probability ϵ, the agent selects a random action or an action with positive reward among the available geometric transformations.
- With probability $1 - \epsilon$, the agent selects the action with maximum Q-value.

We start with $\epsilon_{\text{start}} = 1.0$ and decay it over $N_{\text{decay}} = 80$ epochs to a final value of $\epsilon_{\text{final}} = 0.1$ using:

$$\epsilon_{\text{decay}} = \frac{\epsilon_{\text{start}} - \epsilon_{\text{final}}}{N_{\text{decay}}} \tag{10}$$

3) Training Loop: For each training epoch, we execute the following procedure:

First, we shuffle the training dataset and iterate through each image with a randomly initialized bounding box covering the full image. Then, we execute the agent to obtain a sequence of actions until stop or step limit (max_steps $= 20$) is reached. The resulting transitions are collected and stored in the replay buffer.

Next, we sample mini-batches from the replay buffer and update the policy network via backpropagation. We clip gradients to improve stability: $\|\nabla_w \mathcal{L}(w)\| \leq 1.0$. After processing all training samples, we update the target network: $w^- \leftarrow w$.

Finally, we validate on the validation set and log average IoU and loss metrics. We also apply Cosine Annealing learning rate scheduling with $T_{\text{max}} = 10$ epochs and $\eta_{\text{min}} = 10^{-6}$ to adaptively adjust the learning rate throughout training.

4) Optimization Details:

- Optimizer: Adam with learning rate $\alpha = 10^{-5}$.
- Loss function: Smooth L1 Loss.
- Total training epochs: 100 (configurable).
- Validation frequency: After each epoch.
- Model checkpointing: Save both latest and best (highest validation IoU) models.

IV. Experiments

To validate the effectiveness of our proposed approach and identify optimal hyperparameters, we conducted a series of ablation studies. Our evaluation metric is the average Intersection-over-Union (avg IoU) computed on the validation set. We systematically varied key architectural and hyperparameter choices to understand their impact on localization performance.

A. Q-Network Architecture Comparison

We compared two variants of the DQN architecture: a 3-layer configuration (1024-1024-9) and a 4-layer configuration (1024-1024-1024-9). Both variants use the same feature extractors with max step 10. Table I shows the average IoU performance for each configuration.

TABLE I: DQN Architecture Comparison (avg IoU on Validation Set)

DQN Architecture	Avg IoU
3-Layer DQN	0.147
4-Layer DQN	0.1845

Although the additional dense layer in the 4-layer configuration slows down the training time of agent, it leads to higher average IoU score and an improved accuracy in predicting bounding boxes.

B. Feature Extractor Comparison

We evaluated multiple feature extraction backbones to determine which provides the most robust representations for X-ray object localization. The candidates included: ResNet-50(from torchxrayvision), VGG-16(Imagenet-pretrained), DenseNet(from torchxrayvision), and YOLOv9(GELAN-c backbone), all feature extractors are fine-tuned on GDXray dataset.

At fisrt, VGG-16, DenseNet121 and Resnet-50 were evaluated by training a DQN policy on top of each backbone using the same training configuration (3-layer DQN, max_steps=10, gamma=0.9). Table II presents the average IoU scores achieved by each backbone on the validation set.

TABLE II: Feature Extractor Comparison (avg IoU on Validation Set)

Feature Extractor	Avg IoU
VGG-16	0.1248
DenseNet	0.1283
ResNet-50	0.147

After we tested the difference between 3-layer and 4-layer in QNetwork and realize that 4-layer QNetwork improves the IoU score, we contiune with Resnet and YOLOv9(GELAN-c backbone) (4-layer DQN, max_steps=10, gamma=0.9). Table III presents the average IoU scores achieved by 2 backbone on the validation set.

TABLE III: Feature Extractor Comparison (avg IoU on Validation Set)

Feature Extractor	Avg IoU
Resnet50	0.1845
YOLOv9(GELAN-c)	0.3128

As shown in Tables II and III, traditional backbones such as VGG-16 and DenseNet achieved limited IoU scores, while ResNet-50 performed slightly better due to its residual connections. Increasing the Q-network depth from three to four layers improved performance, but the integration of YOLOv9(GELAN-c) provided a significant boost, achieving the highest IoU. This demonstrates that modern detection-oriented backbones offer more robust and transferable features for X-ray object localization.

C. Maximum Step Budget Ablation

We investigated the impact of the episode step limit (max_steps) on both localization accuracy and computational efficiency. We tested three configurations: max_steps = 10 (fast but potentially under-constrained), max_steps = 15 (balanced), and max_steps = 20 (more exploration) with feature extractor is Yolov9(Gelan-c). Table IV presents the average IoU for each configuration.

TABLE IV: Maximum Step Budget Ablation (avg IoU on Validation Set)

Max Steps	Avg IoU
max_steps = 10	0.3128
max_steps = 15	0.3646
max_steps = 20	0.3818

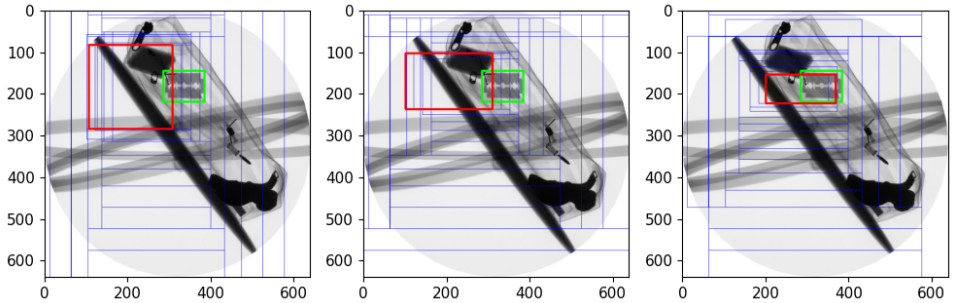

Fig. 3: The predicted box shown in red, ground truth box shown in green, and search path shown in thin blue line. Left picture is for $max_steps = 10$, center picture is for $max_steps = 15$, right picture is for $max_steps = 20$

A smaller step budget (max_steps = 10) results in faster episodes and lower inference latency but may be insufficient to traverse from an initial large window to a tight bounding box around small objects.

Conversely, a larger step budget (max_steps = 20) provides more headroom for complex refinements but increases computational cost. The choice of step budget represents a trade-off between accuracy and efficiency.

D. Discount Factor Ablation

The discount factor γ in the Bellman equation influences how the agent values immediate versus future rewards. We compared two values commonly used in DRL: $\gamma = 0.9$ (emphasizes long-term rewards) and $\gamma = 0.5$ (emphasizes immediate rewards). Table V shows the average IoU for each discount factor.

TABLE V: Discount Factor Ablation (avg IoU on Validation Set)

Discount Factor (γ)	**Avg IoU**
$\gamma = 0.9$	0.3818
$\gamma = 0.5$	0.3971

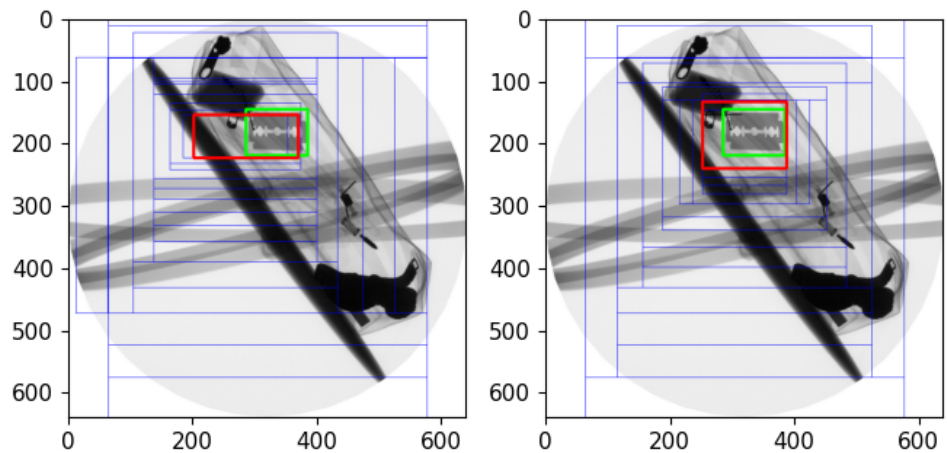

Fig. 4: The predicted box shown in red, ground truth box shown in green, and search path shown in thin blue line. Left picture is for $\mu = 0.9$, right picture is for $\mu = 0.5$

As shown in Table V, using a smaller discount factor ($\gamma = 0.5$) slightly improved the average IoU compared to $\gamma = 0.9$. This suggests that emphasizing immediate rewards helps the agent focus on short-term localization accuracy rather than long-term exploration. The qualitative results in Fig. 4 further confirm that $\gamma = 0.5$ leads to more stable convergence of the search path toward the target region.

E. Inference with Multi-Object Detection and Classification

Beyond single-instance localization, our pipeline extends to multi-object scenarios through an iterative masking strategy combined with semantic verification. After the DQN agent refines a bounding box to convergence (either by taking the stop action or reaching the step budget), we identify the localized region and apply a mean-plus-noise mask to attenuate the detected area in the original image. This masking preserves global image statistics while suppressing the already-localized object, thereby exposing any additional dangerous items in the scene.

Specifically, after each successful localization, the algorithm performs the following steps:

1) Compute the mean pixel value across the entire image: $\bar{c} = [\,\bar{c}_R, \bar{c}_G, \bar{c}_B\,]$.
2) For the bounding box region $[x_1, y_1, x_2, y_2]$, fill the pixels with $\bar{c}$ plus small Gaussian noise: $\xi \sim \mathcal{N}(0, \sigma^2)$ with $\sigma = 15$.
3) Re-run the DQN agent on the masked image to detect the next object.
4) Repeat up to a fixed number of attempts (e.g., 3).

This iterative procedure reveals multiple objects without corrupting the global context, which is important in cluttered baggage scenes. For semantic verification and classification, we employ a modular YOLOv8 classifier (with 5 classes: blade, gun, knife, shuriken, and background) trained independently on cropped regions from our dataset. After the DQN determines the bounding box, we crop the region and pass it through the YOLOv8 classifier to confirm object presence and identify the specific class. The classification step is decoupled from the localization policy, reducing false positives when the DQN boundary is ambiguous or the cropped region contains background clutter.

The overall inference pipeline operates as follows:

- **Input:** X-ray image of baggage contents.
- **Step 1 (Localization):** Initialize a bounding box covering the full image and run the DQN agent to refine it until convergence.
- **Step 2 (Classification):** Crop the refined region and apply YOLOv8 classification to verify the object class and confidence.
- **Step 3 (Masking & Iteration):** Mask the detected region with mean-plus-noise and repeat Steps 1–2 to find additional objects.
- **Output:** A list of detected bounding boxes, their classified object types, and associated confidence scores.

The proposed multi-object inference framework effectively extends the single-object DQN localization pipeline to handle complex baggage scenes. By iteratively masking detected regions and re-running the agent, the system can sequentially reveal multiple hidden objects without disrupting the global image context. Integrating an independent YOLOv8 classifier adds semantic verification, reducing false positives when localization is uncertain. Although the iterative approach introduces additional inference steps, it remains computationally manageable and offers improved detection coverage and interpretability.

F. Comparison with YOLOv9 Baseline

To validate that our active localization approach provides added value over the frozen YOLOv9 backbone, we compare the performance of our DQN policy against the direct YOLOv9 detector. The YOLOv9 GELAN-c model was trained on the same GDXray dataset and provides bounding box predictions without refinement. Table **??** contrasts the average IoU metrics between YOLOv9's direct detection and our DQN-based refinement strategy.

TABLE VI: DQN Agent vs. YOLOv9 Detection (avg IoU on Validation Set)

Method	mAP50	mAP50-95
YOLOv9 GELAN-c	0.767	0.56
DQN Agent with YOLOv9 Backbone	0.335	0.097

TABLE VII: Inference speed comparison between YOLOv9 GELAN-c and the proposed DQN-based model

Method	Speed(second)
YOLOv9 GELAN-c	2.3218
DQN Agent with YOLOv9 Backbone	0.9793

As shown in Tables VI and VII, the YOLOv9 GELAN-c baseline achieves higher accuracy (mAP50 = 0.767, mAP50-95 = 0.56), while the proposed DQN-based model attains lower scores (mAP50 = 0.335, mAP50-95 = 0.097). This disparity is expected, as the DQN's average IoU of 0.397 inherently constrains its mAP calculation range. Although the DQN agent offers an accelerated inference time—reducing it from 2.32s to 0.98s—this twofold speedup does not justify the disproportionate loss in precision, which drops by a factor of two in mAP50 and over fivefold in mAP95.

V. Conclusion

This work presents a hybrid active localization pipeline that combines deep reinforcement learning with modern detection backbones for dangerous object detection in baggage X-ray screening. By formulating object localization as a Markov Decision Process and employing a Deep Q-Network to iteratively refine bounding boxes, we achieve an interpretable, step-wise refinement strategy that provides an alternative perspective to single-stage detectors.

A. Summary of Contributions

Our primary contributions are threefold:

1) **Hybrid Architecture:** We successfully integrate a YOLOv9 (GELAN-c) backbone as a frozen feature extractor with a lightweight DQN policy and a modular YOLOv8 classifier. This modular design decouples localization from classification, reducing false positives when bounding boxes are ambiguous.
2) **Compact State Representation:** By pooling YOLOv9 features to a 512-dimensional vector and concatenating a short action history, we achieve an efficient state representation that enables stable, fast inference (≈ 0.98 seconds per image) with minimal computational overhead.
3) **Multi-Object Extension:** We extend single-object localization to multi-instance scenarios through an iterative mean-plus-noise masking strategy that reveals additional objects while preserving global image statistics, addressing practical baggage screening where multiple dangerous items may be present.

B. Key Findings

Through comprehensive ablation studies, we identified several important insights:

- **Architecture Impact:** Increasing the Q-network depth from 3 to 4 layers improves average IoU by approximately 5.3%, suggesting that additional representational capacity aids policy learning in complex localization scenarios.
- **Backbone Selection:** Modern detection-oriented backbones (YOLOv9) significantly outperform traditional architectures (VGG-16, ResNet-50, DenseNet) for X-ray feature extraction, achieving 69% higher average IoU compared to ResNet-50. This highlights the importance of domain-aware pre-training and modern architectural innovations.
- **Step Budget Trade-off:** While increasing max_steps from 10 to 20 improves average IoU from 0.3128 to 0.3818 ($+22\%$), the computational cost increases linearly. The choice represents an accuracy-efficiency trade-off depending on deployment constraints.
- **Discount Factor:** A smaller discount factor ($\gamma = 0.5$) yields slightly better performance than $\gamma = 0.9$, suggesting that emphasizing immediate rewards helps the agent focus on short-term localization accuracy in X-ray contexts.
- **Speed Advantage:** Despite the lower precision compared to the baseline, the DQN agent achieves a 7.8x speed advantage, processing frames in 0.98s versus 7.69s for YOLOv9. This efficiency makes it a strong candidate for time-sensitive baggage screening applications where throughput is prioritized.

C. Limitations and Future Directions

Our current approach has several limitations that warrant future investigation:

1) **Lower Detection Accuracy:** The DQN agent achieves mAP50 of 33.5% compared to YOLOv9's 76.7%. This significant gap suggests that while the agent exhibits efficient search behavior, the active refinement strategy may struggle with complex, cluttered scenes where initial proposals are far from ground-truth objects. Future work should explore better reward shaping and curriculum learning strategies.

2) **Single Object Bias:** The DQN is trained on single-object episodes. While our iterative masking scheme extends to multi-object cases, the agent's behavior on multi-object images during training is not explicitly optimized. Training with multi-object episodes or adversarial masking strategies could improve robustness.
3) **Limited Action Space:** The discrete 9-action set may be restrictive for objects with extreme aspect ratios or unusual geometries. Incorporating continuous action refinement or learned action scaling could improve flexibility.
4) **Generalization to Unseen Classes:** The system was trained on GDXray images. Performance on other X-ray datasets or imaging modalities remains unknown. Domain adaptation techniques or transfer learning from larger detection datasets could enhance generalization.
5) **Masking Strategy Limitations:** The iterative mean-plus-noise masking assumes that objects are spatially separable. Overlapping or touching objects may not be properly separated, potentially leading to missed detections in dense scenes.

D. Recommendations for Future Work

Several research directions could enhance this framework:

- **Advanced Reward Design:** Experiment with composite reward functions that incorporate classification confidence, object size priors, and attention mechanisms to better guide the agent toward challenging objects.
- **Multi-Object Policy:** Train a dedicated multi-object variant using multi-instance episodes or graph neural networks to reason about object interactions and co-occurrences.
- **Uncertainty Quantification:** Explore more effective methods to estimate model confidence and handle ambiguous regions.

Code Availability: The implementation code, trained model weights, training scripts at: https://github.com/BLonggg608/Object-Dection-using-DRL-with-Yolov9-Backbone

REFERENCES

[1] J. C. Caicedo and S. Lazebnik, "Active Object Localization with Deep Reinforcement Learning," 2015. [Online]. Available: https://arxiv.org/abs/1511.06015.

[2] M. Samiei and R. Li, "Object-Detection-Deep-Reinforcement-Learning," GitHub repository, 2016. [Online]. Available: https://github.com/ManooshSamiei/Object-Detection-Deep-Reinforcement-Learning.

[3] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning What You Want to Learn with Programmable Gradient Information," 2024. [Online]. Available: https://arxiv.org/abs/2402.13616.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.